ClamAV S3 Scanner Project Guide

This PDF explains the whole repository in beginner-friendly language. It covers what the project does, how the folders fit together, and what each tracked file is responsible for.

What The Project Does

This repository builds an AWS upload pipeline that scans files for viruses with ClamAV.

1. A client calls an API to ask for a temporary upload URL.
2. A Lambda function creates a presigned S3 upload URL.
3. The client uploads a file to an S3 bucket.
4. S3 sends an event to an SQS queue.
5. An ECS Fargate container reads the queue message.
6. The container downloads the file and runs ClamAV.
7. The file is copied into either a clean bucket or an infected bucket.
8. The original object is tagged, and can optionally be deleted.

> Important structure note: `bussness_severes/` contains deployable Terraform stacks, `terraform_modules/` contains reusable infrastructure modules, and `containers/` contains the scanner application source code. There is no root `main.tf`, so Terraform is intended to run from the stack folders.

Root And Environment Files

- `.gitignore` : Empty. Git is not currently ignoring Terraform state, plans, or secrets.
- `terraform.tfvars` : Example root variable file with `aws_region`, `environment`, and `resource_prefix`. It is a sample input file, not the main deployment entry point.
- `environments/dev-bushan.tfvar` : Environment-specific values for `environment` and `resource_prefix`. Because the extension is `.tfvar` instead of `.tfvars`,

Terraform would normally need it passed explicitly with `-var-file`.

Deployable Terraform Stacks

bussness_severes/clamav-s3-scanner

- `bussness_severes/clamav-s3-scanner/provider.tf` : Declares the AWS provider and pins it to the `hashicorp/aws` 5.x line.
- `bussness_severes/clamav-s3-scanner/variable.tf` : Defines all inputs for the full solution, including region, environment, naming prefix, VPC CIDR, scanner image, ECS CPU and memory, task count, and whether the original uploaded file should be deleted.
- `bussness_severes/clamav-s3-scanner/main.tf` : Main infrastructure entry point. It wires together four modules: upload API, VPC, ECS cluster, and scanner service. This is the best first file to read.
- `bussness_severes/clamav-s3-scanner/output.tf` : Exports the upload API URL, upload bucket name, clean bucket name, infected bucket name, and scan queue URL after deployment.
- `bussness_severes/clamav-s3-scanner/terraform.tfvars.example` : Example values for the full stack, including the required `scanner_image` setting.

bussness_severes/api-gateway-s3-upload

- `bussness_severes/api-gateway-s3-upload/provider.tf` : AWS provider declaration for the smaller upload-only stack.
- `bussness_severes/api-gateway-s3-upload/variable.tf` : Defines inputs like environment, resource prefix, region, and CIDR block. Not all of those values are used by the stack as written.
- `bussness_severes/api-gateway-s3-upload/main.tf` : Deploys only the upload API module and hardcodes a region and resource prefix. It looks like a smaller or older stack focused only on presigned upload URLs.
- `bussness_severes/api-gateway-s3-upload/output.tf` : Empty placeholder file with no Terraform outputs.
- `bussness_severes/api-gateway-s3-upload/terraform.tfstate` : Generated Terraform state snapshot for that stack.
- `bussness_severes/api-gateway-s3-upload/terraform.tfstate.backup` : Backup copy of that same state snapshot.

bussness_severes/gitlab_runners

- `bussness_severes/gitlab_runners/main.tf` : Partial or experimental stack. It currently creates only the VPC module, while the ECS cluster section is commented out.

Reusable Terraform Modules

terraform_modules/api-s3-connector

- `terraform_modules/api-s3-connector/variable.tf` : Module input definition. The main value here is `resource_prefix` .
- `terraform_modules/api-s3-connector/data.tf` : Reads the current AWS region so it can be passed into Lambda.
- `terraform_modules/api-s3-connector/main.tf` : Creates the upload S3 bucket, configures browser CORS, creates the Lambda IAM role and policy, zips the Lambda source, deploys the Lambda function, and grants API Gateway permission to invoke it.
- `terraform_modules/api-s3-connector/api-gateway.tf` : Creates an HTTP API Gateway, integrates it with Lambda using AWS proxy mode, adds route `GET /upload-url` , and auto-deploys the default stage.
- `terraform_modules/api-s3-connector/output.tf` : Exports the upload bucket name and ARN, API endpoint, and Lambda function name.
- `terraform_modules/api-s3-connector/lambda_function.py` : Python Lambda code. It reads a `filename` query parameter, sanitizes it with `os.path.basename` , prefixes it with `uploads/` , generates a presigned `put_object` URL, and returns JSON containing the upload URL and object key.
- `terraform_modules/api-s3-connector/function.zip` : Generated Lambda deployment package built from `lambda_function.py` .
- `terraform_modules/api-s3-connector/response.json` : Saved example of a successful Lambda response containing a temporary upload URL and key.
- `terraform_modules/api-s3-connector/hello.txt` : Tiny sample file, likely kept for manual upload tests.
- `terraform_modules/api-s3-connector/terraform.tfstate` : Generated Terraform state for this module's standalone deployment.
- `terraform_modules/api-s3-connector/terraform.tfstate.backup` : Backup state snapshot for the same module.

terraform_modules/clamav-s3-scanner

- `terraform_modules/clamav-s3-scanner/variable.tf` : Defines the scanner module inputs, including source bucket, network IDs, ECS cluster ARN, scanner image, task sizing, object prefix, and delete behavior.
- `terraform_modules/clamav-s3-scanner/main.tf` : Core infrastructure module. It creates the clean and infected buckets, public-access blocks, scan queue, dead-letter queue, queue policy, source bucket notification, log group, security group, ECS IAM roles, ECS task definition, and ECS service.
- `terraform_modules/clamav-s3-scanner/output.tf` : Exports the clean and infected bucket names, scan queue URL and ARN, ECS service name, and CloudWatch log group name.

The scanner module also contains the important policy logic that lets S3 send events to SQS and lets the ECS task read from SQS and move objects among the source, clean, and infected buckets.

terraform_modules/ecs_cluster

- `terraform_modules/ecs_cluster/variable.tf` : Inputs for the ECS cluster module: cluster name, container insights flag, and tags.
- `terraform_modules/ecs_cluster/main.tf` : Creates one ECS cluster and conditionally enables CloudWatch Container Insights.
- `terraform_modules/ecs_cluster/output.tf` : Exports the cluster ARN and cluster name.
- `terraform_modules/ecs_cluster/terraform.tfstate` : Generated Terraform state for this module.

terraform_modules/vpc

- `terraform_modules/vpc/variable.tf` : Inputs for the VPC module: resource prefix, environment, and VPC CIDR block.
- `terraform_modules/vpc/site.tf` : Helper logic for the network. It reads available availability zones, keeps the first two, calculates subnet CIDRs with `cidrsubnet` , and builds a shared name prefix.
- `terraform_modules/vpc/main.tf` : Creates the VPC, two public subnets, an internet gateway, a public route table, a default internet route, and route-table associations.
- `terraform_modules/vpc/output.tf` : Exports the VPC ID and public subnet IDs.
- `terraform_modules/vpc/terraform.tfstate` : Generated Terraform state for the VPC module.

Container Application

- `containers/clamav-s3-scanner/Dockerfile` : Builds the scanner image from `python:3.11-slim-bookworm` , installs ClamAV packages, installs Python dependencies, copies the app files, prepares local directories, and sets the container entrypoint.
- `containers/clamav-s3-scanner/requirements.txt` : Python dependency list. It currently contains only `boto3` , the AWS SDK used to talk to S3 and SQS.
- `containers/clamav-s3-scanner/entrypoint.sh` : Startup shell script. It runs `freshclam` to refresh virus definitions, prints a warning if that fails, then starts the Python worker.
- `containers/clamav-s3-scanner/scanner.py` : Main runtime worker. It reads configuration from environment variables, refreshes virus definitions on a timer, polls SQS with long polling, parses the embedded S3 event records, downloads the file, runs `clamscan` , copies the object into the clean or infected bucket, tags the source object, optionally deletes the original, and deletes the queue message only after successful processing.

Why `scanner.py` matters most: it is the part of the system that actually decides whether a file is clean or infected. Terraform creates the AWS plumbing, but this file is the runtime engine.

Generated Artifacts And State Files

These files are important for deployment history, but they are not handwritten application logic and usually should not be edited manually.

- `terraform_modules/api-s3-connector/function.zip` : Generated Lambda zip package.
- `terraform_modules/api-s3-connector/terraform.tfstate` : State snapshot for the upload API module.
- `terraform_modules/api-s3-connector/terraform.tfstate.backup` : Backup snapshot for that module.
- `terraform_modules/ecs_cluster/terraform.tfstate` : State snapshot for the ECS cluster module.

- `terraform_modules/vpc/terraform.tfstate` : State snapshot for the VPC module.
- `bussness_severes/api-gateway-s3-upload/terraform.tfstate` : State snapshot for the upload-only stack.
- `bussness_severes/api-gateway-s3-upload/terraform.tfstate.backup` : Backup of that stack state.

Best Reading Order For A Beginner

1. `bussness_severes/clamav-s3-scanner/main.tf` : shows how the full system is assembled.
2. `terraform_modules/api-s3-connector/main.tf` : shows how uploads are enabled.
3. `terraform_modules/clamav-s3-scanner/main.tf` : shows how scan jobs, buckets, IAM, and ECS are wired together.
4. `terraform_modules/api-s3-connector/lambda_function.py` : shows how presigned upload URLs are generated.
5. `containers/clamav-s3-scanner/scanner.py` : shows how the actual virus scan worker behaves.

If those five files make sense, the rest of the repository will become much easier to understand.